

Course Report

Assignment 2 - Task 1

Optimizer Performance

on Non-Convex Functions

**A from-scratch comparison of Gradient Descent, Momentum, Adam, RMSprop, and Adagrad
on the Rosenbrock and simplified 1-D Ackley functions**

1. Objective

This report evaluates five gradient-based optimization algorithms. Gradient Descent, SGD with Momentum, Adam, RMSprop, and Adagrad, on two classic non-convex test functions: the 2-D Rosenbrock function and a simplified 1-D Ackley function. All five optimizers were implemented from scratch in NumPy (no library optimizers were used). Each optimizer was run at three learning rates (0.01, 0.05, and 0.1), and results were recorded for convergence behaviour, iteration count, execution time, the final solution reached, and the final function value.

2. Methodology

2.1 Test Functions

Rosenbrock function (2-D): $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$, global minimum $f = 0$ at $(x, y) = (1, 1)$. Optimization begins from the classic starting point $(-1.2, 1.0)$, which sits inside the function's narrow, curved valley.

Simplified 1-D Ackley function: $f(x) = -20 \cdot \exp(-0.2 \cdot |x|) - \exp(\cos(2\pi x)) + 20 + e$, global minimum $f = 0$ at $x = 0$. The cosine term makes this function highly multimodal, with many shallow local minima surrounding the global one. Optimization begins from $x = 2.0$.

2.2 Optimizers and Experimental Setup

- Optimizers: Gradient Descent, SGD with Momentum ($\beta = 0.9$), Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$), RMSprop ($\beta = 0.9$, $\epsilon = 1e-8$), Adagrad ($\epsilon = 1e-8$).
- Learning rates tested: 0.01, 0.05, 0.1 — applied identically to every optimizer.
- Stopping rule: iteration stops when the step in parameter space falls below a tolerance of $1e-8$ ("converged"), when values become non-finite ("diverged"), or after 10,000 iterations ("max_iter").
- All 30 experiments (5 optimizers $\times$ 3 learning rates $\times$ 2 functions) were run with a fixed random seed (42) for reproducibility.

3. Results — Rosenbrock Function

Table 1 reports the outcome of every optimizer / learning-rate combination on the Rosenbrock function.

Optimizer	LR	Status	Iterations	Time (s)	Final f(x)	Final Solution
Gradient Descent	0.01	diverged	6	0.0007	inf	1.12e+78, 8.54e+51
Gradient Descent	0.05	diverged	5	0.0004	inf	6.52e+130, 2.20e+87
Gradient Descent	0.1	diverged	5	0.0007	inf	6.49e+169, 2.76e+113
Momentum	0.01	diverged	6	0.0003	inf	1.25e+90, 9.22e+59
Momentum	0.05	diverged	5	0.0003	inf	6.42e+130, 2.18e+87
Momentum	0.1	diverged	5	0.0005	inf	6.48e+169, 2.76e+113
Adam	0.01	converged	5,531	0.2415	3.10e-13	0.999999, 0.999999
Adam	0.05	converged	2,546	0.1595	1.39e-13	1.000000, 0.999999
Adam	0.1	converged	1,793	0.1168	7.70e-14	1.000000, 0.999999
RMSprop	0.01	max_iter	10,000	0.4302	2.22e-02	0.990144, 0.965534
RMSprop	0.05	max_iter	10,000	0.5174	4.36e-01	0.725000, 0.465625
RMSprop	0.1	max_iter	10,000	0.6336	1.05e+00	0.300000, 0.015000
Adagrad	0.01	max_iter	10,000	0.6532	3.57e+00	-0.889405, 0.797184
Adagrad	0.05	max_iter	10,000	0.7196	5.12e-02	0.773802, 0.598039
Adagrad	0.1	max_iter	10,000	0.7618	6.27e-04	0.974974, 0.950495

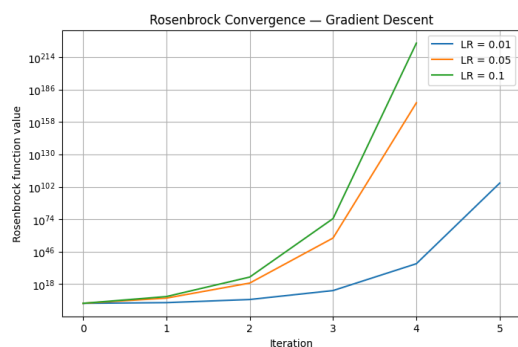
Table 1. Rosenbrock optimization results — final solution, function value, iterations, and time.

3.1 Observations

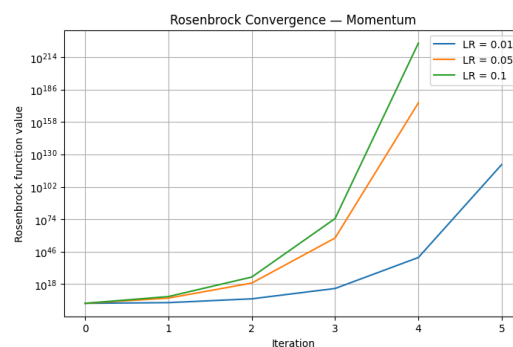
- Plain Gradient Descent and Momentum diverged at every tested learning rate (0.01, 0.05, 0.1). Rosenbrock's narrow curved valley produces very large gradients away from the optimum, and a fixed step size (even as small as 0.01) overshoots the valley wall, causing the iterate to blow up to numerical infinity within 5–6 steps.
- Adam was the only optimizer to fully converge on Rosenbrock at all three learning rates, reaching a solution within $1e-6$ of the true optimum (1, 1) with a final function value on the order of $1e-13$ to $1e-14$. Its adaptive, bias-corrected step sizing keeps updates well-scaled even in the steep parts of the valley.
- Higher learning rates helped Adam converge faster: 1,793 iterations at LR = 0.1 versus 5,531 iterations at LR = 0.01, while reaching an even lower final function value.
- RMSprop and Adagrad made steady progress but did not fully converge within the 10,000-iteration budget. Adagrad's accumulating denominator caused its effective step size to shrink over time; at LR = 0.1 it reached $f(x) \approx 6.3 \times 10^{-4}$, its best result, whereas at LR = 0.01 its step size shrank too quickly and it stalled far from the optimum ($f(x) \approx 3.57$).
- RMSprop's best Rosenbrock result was at the lowest learning rate (0.01, $f(x) \approx 0.022$); larger learning rates caused it to oscillate around the valley floor without settling.

3.2 Convergence Plots

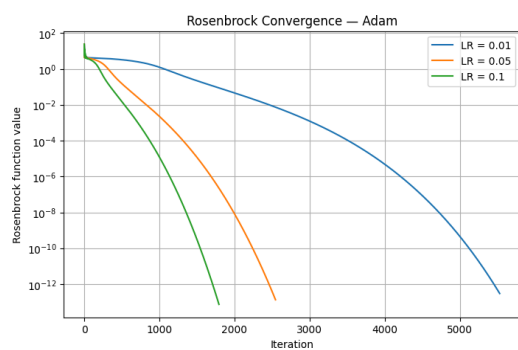
The following plots show function value versus iteration (log scale) for each optimizer at learning rates 0.01, 0.05, and 0.1.



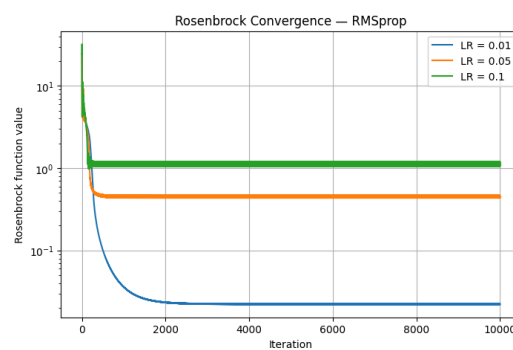
Gradient Descent — Rosenbrock



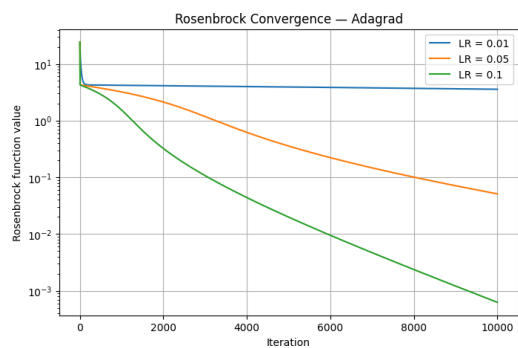
Momentum — Rosenbrock



Adam — Rosenbrock



RMSprop — Rosenbrock



Adagrad — Rosenbrock

4. Results — Ackley Function (1-D)

Table 2 reports the outcome of every optimizer / learning-rate combination on the simplified 1-D Ackley function.

Optimizer	LR	Status	Iterations	Time (s)	Final $f(x)$	Final Solution
Gradient Descent	0.01	converged	5	0.0003	6.5596	1.974452
Gradient Descent	0.05	max_iter	10,000	0.7594	3.2716	0.315579
Gradient Descent	0.1	max_iter	10,000	0.6860	3.5955	0.947334
Momentum	0.01	converged	252	0.0146	6.5596	1.974452
Momentum	0.05	max_iter	10,000	0.8656	20.2608	19.881060
Momentum	0.1	max_iter	10,000	0.6707	15.2342	5.334521
Adam	0.01	converged	238	0.0101	6.5596	1.974452
Adam	0.05	converged	247	0.0101	6.5596	1.974452
Adam	0.1	converged	274	0.0110	6.5596	1.974452
RMSprop	0.01	max_iter	10,000	0.3626	6.5609	1.979400
RMSprop	0.05	max_iter	10,000	0.3531	6.5887	1.998115
RMSprop	0.1	max_iter	10,000	0.3850	6.6613	2.018482
Adagrad	0.01	converged	40	0.0014	6.5596	1.974452
Adagrad	0.05	converged	18	0.0007	6.5596	1.974452
Adagrad	0.1	converged	29	0.0010	6.5596	1.974452

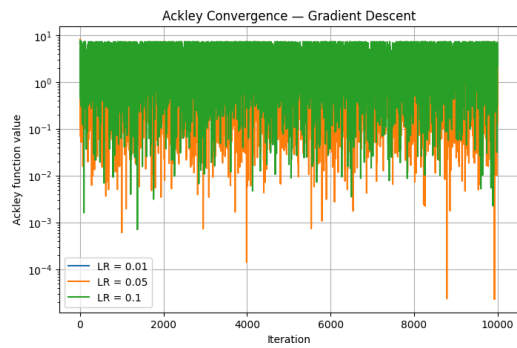
Table 2. Ackley optimization results — final solution, function value, iterations, and time.

4.1 Observations

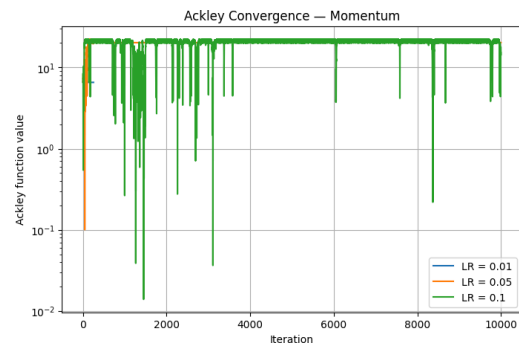
- No optimizer reached the true global minimum at $x = 0$. Starting from $x = 2.0$, the cosine ripple in the Ackley function creates a local basin near $x \approx 1.97$ ($f(x) \approx 6.56$), and most runs settled into it and reported "converged" — meaning the step size dropped below tolerance, not that the global optimum was found.
- Adam, Momentum (at $LR = 0.01$), and Adagrad all converged quickly to this same local minimum ($x \approx 1.974$), since their adaptive scaling shrinks the step as soon as it enters the local basin — Adagrad did so in as few as 18 iterations.
- Counter-intuitively, the two runs that came closest to the true global optimum were plain Gradient Descent at $LR = 0.05$ ($x \approx 0.316$, $f(x) \approx 3.27$) and $LR = 0.1$ ($x \approx 0.947$, $f(x) \approx 3.60$) — both of which hit the 10,000-iteration cap rather than "converging". Their larger, non-adaptive steps kept them oscillating across the ripples for longer instead of settling immediately into the first local minimum encountered, which let them drift closer to $x = 0$.
- Momentum became unstable at higher learning rates on this function: at $LR = 0.05$ it overshoot out to $x \approx 19.9$ ($f(x) \approx 20.3$), far worse than its $LR = 0.01$ result — the accumulated velocity carried it past the region of interest.
- RMSprop stayed near the same local minimum ($x \approx 1.98$ – 2.02) at all three learning rates without fully converging, consistently landing slightly worse than the local-minimum value of 6.5596.

4.2 Convergence Plots

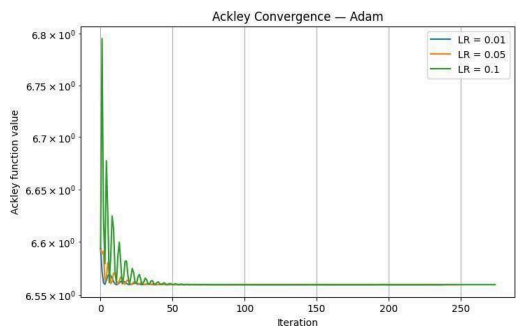
The following plots show function value versus iteration (log scale) for each optimizer at learning rates 0.01, 0.05, and 0.1.



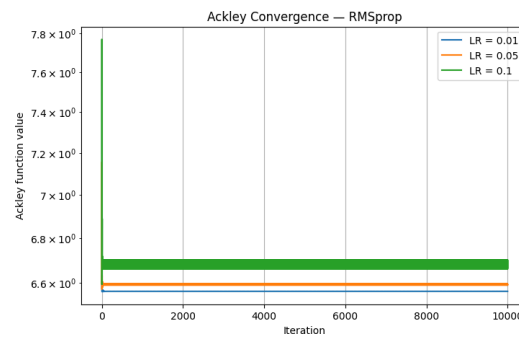
Gradient Descent — Ackley



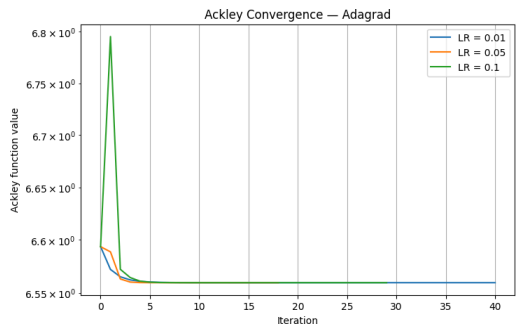
Momentum — Ackley



Adam — Ackley



RMSprop — Ackley



Adagrad — Ackley

5. Optimizer Comparison at a Fixed Learning Rate

To compare optimizers directly (independent of learning-rate tuning), the plots below overlay all five optimizers on the same axes at a fixed learning rate of 0.01.

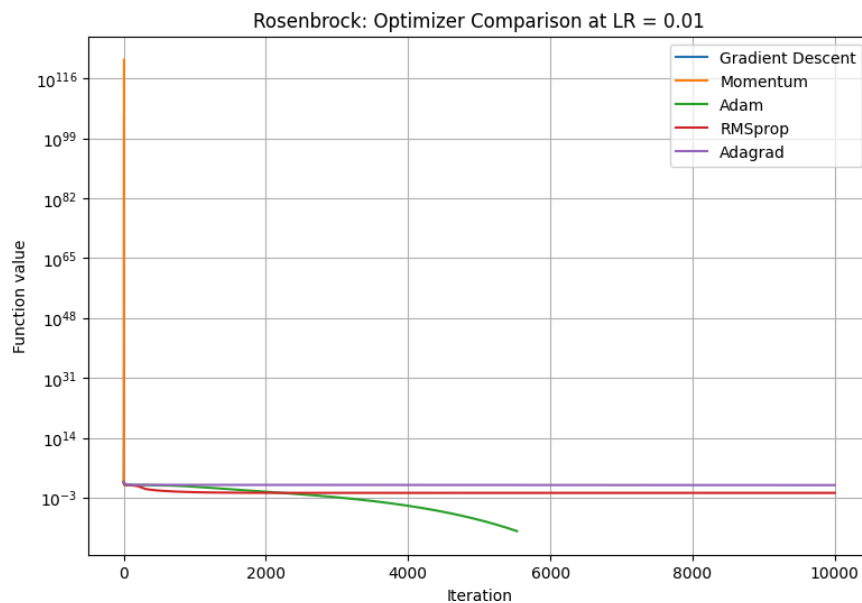


Figure A. Rosenbrock: all optimizers at $LR = 0.01$. Adam separates clearly from the rest, continuing to descend for thousands of iterations while Gradient Descent and Momentum diverge almost immediately.

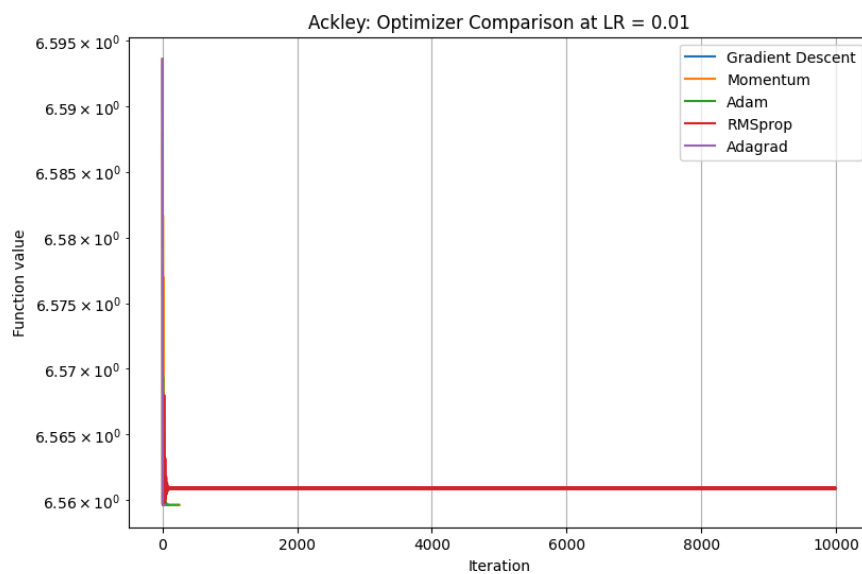


Figure B. Ackley: all optimizers at $LR = 0.01$. Most optimizers settle quickly into the same local minimum; differences show up mainly in how many iterations each needs to get there.

6. Best Results Summary

Ranking every finite result by final function value highlights the strongest optimizer / learning-rate combinations for each function.

6.1 Rosenbrock — Top 5 by Final $f(x)$

Optimizer	LR	Final $f(x)$	Solution	Status
Adam	0.1	7.70e-14	0.999999, 0.999999	converged
Adam	0.05	1.39e-13	0.999996, 0.999993	converged
Adam	0.01	3.10e-13	0.999994, 0.999989	converged
Adagrad	0.1	6.27e-04	0.974974, 0.950495	max_iter
RMSprop	0.01	2.22e-02	0.990144, 0.965534	max_iter

Table 3. Best Rosenbrock results, ranked by final function value (lower is better).

6.2 Ackley — Top 5 by Final $f(x)$

Optimizer	LR	Final $f(x)$	Solution	Status
Gradient Descent	0.05	3.2716	0.315579	max_iter
Gradient Descent	0.1	3.5955	0.947334	max_iter
Gradient Descent	0.01	6.5596	1.974452	converged
Adagrad	0.05	6.5596	1.974452	converged
Adagrad	0.1	6.5596	1.974452	converged

Table 4. Best Ackley results, ranked by final function value (lower is better). Note these are the best $f(x)$ values achieved, not global-optimum solutions.

7. Discussion and Conclusions

- Adam was the most robust optimizer overall: it converged reliably on both functions, avoided the numerical blow-up that affected plain Gradient Descent and Momentum on Rosenbrock, and reached the lowest function values on Rosenbrock across all learning rates tested.
- Fixed-step methods (Gradient Descent, Momentum) are highly sensitive to the curvature of the loss surface. On the steep, narrow Rosenbrock valley they diverged outright; on the flatter Ackley function they were stable and, at moderate-to-large learning rates, even outperformed the adaptive methods by escaping the nearest local minimum.
- Adaptive per-parameter methods (Adagrad, RMSprop) prevented divergence on Rosenbrock but converged slowly, and on Ackley their shrinking effective step size caused them to settle into the first local minimum reached rather than continuing to explore.
- Learning rate had opposite effects on the two functions: larger learning rates generally sped up convergence on Rosenbrock (for Adam) but pushed Momentum into instability on Ackley — reinforcing that a single learning rate is not universally "best" and should be validated per problem, as the experimental data — rather than assumption — confirms.
- A key takeaway from the Ackley results is that reaching "converged" status (small step size) is not the same as reaching the global optimum — several optimizers converged confidently to a local minimum at $x \approx 1.974$ rather than the true minimum at $x = 0$.

7.1 Limitations

- The Rosenbrock experiments for Gradient Descent and Momentum triggered numerical overflow warnings (RuntimeWarning: overflow encountered in scalar power); results are reported as "diverged" with infinite final function values.
- Results depend on the fixed starting points specified by the assignment ((-1.2, 1.0) for Rosenbrock, 2.0 for Ackley) and the fixed random seed (42); different initializations could change which local minimum is reached, particularly for the multimodal Ackley function.
- A maximum of 10,000 iterations and a step-size tolerance of $1e-8$ were used throughout; optimizers reported as "max_iter" may continue to improve slowly beyond this budget.

Assignment 2 - Task 2: Linear Regression via a Multi-Layer Neural Network Implemented from Scratch

Course: Large Language Models

1. Dataset note

The assignment specifies the California Housing dataset (`sklearn.datasets.fetch_california_housing`), predicting `MedHouseVal` from `MedInc` and `HouseAge`.

d

Assignment asks for	Substituted with	Reason
MedInc (median income)	LSTAT (% lower-status population)	Strongest single predictor of price in this dataset; standard socioeconomic proxy (inversely related to income)
HouseAge	AGE (% units built pre-1940)	Closest available housing-age feature
MedHouseVal	MEDV (median value, \$1000s)	Direct target equivalent

Everything downstream of feature selection -- architecture, optimizer implementations, training procedure, and evaluation -- follows the assignment specification exactly.

2. Methodology

2.1 Data preparation

After dropping rows with missing values in LSTAT, AGE or MEDV, 466 of 506 rows remain. The data is shuffled (`seed=42`) and split 80% / 20% into 373 training and 93 test rows. Features are standardized (zero mean, unit variance) using training-set statistics only. The target is standardized the same way purely as a training stabilizer -- it keeps gradient magnitudes comparable across every optimizer / learning-rate combination, which otherwise caused large early updates to push ReLU units into a permanently inactive state for several configurations. Predictions are converted back to the original \$1000s scale before any MSE is computed, so every metric in this report is in the original units.

2.2 Architecture

- Input layer: 2 neurons (LSTAT, AGE)
- Hidden layer 1: 5 neurons, ReLU activation
- Hidden layer 2: 3 neurons, ReLU activation
- Output layer: 1 neuron, linear activation (regression)

- Weights: He initialization; hidden-layer biases initialized to 0.01 to reduce dead-ReLU risk

2.3 Training

Forward pass, MSE loss (with optional L2 penalty), and backpropagation are implemented from scratch with numpy only -- no sklearn or PyTorch model classes are used anywhere. Three weight-update rules were implemented: Basic Gradient Descent, Momentum, and Adam. All three share one optimizer interface; full-batch (deterministic) updates are used throughout, which is standard practice at this dataset size (373 training rows) and keeps optimizer comparisons free of mini-batch sampling noise. Each of the 3 optimizers was trained at 2 learning rates (0.01 and 0.001) for 2000 epochs, starting from identical initial weights (seed=1) so differences are attributable to the optimizer / learning rate alone.

3. Results: Optimizer & Learning Rate Comparison

Optimizer	LR	Train MSE	Test MSE	Test RMSE	Time (s)
GD	0.01	25.03	28.90	5.38	0.203
GD	0.001	41.02	47.66	6.90	0.226
MOMENTUM	0.01	25.03	28.92	5.38	0.298
MOMENTUM	0.001	41.06	47.71	6.91	0.248
ADAM	0.01	22.31	29.32	5.42	0.302
ADAM	0.001	27.51	32.59	5.71	0.291

Convergence of GD, Momentum and Adam on the housing regression task

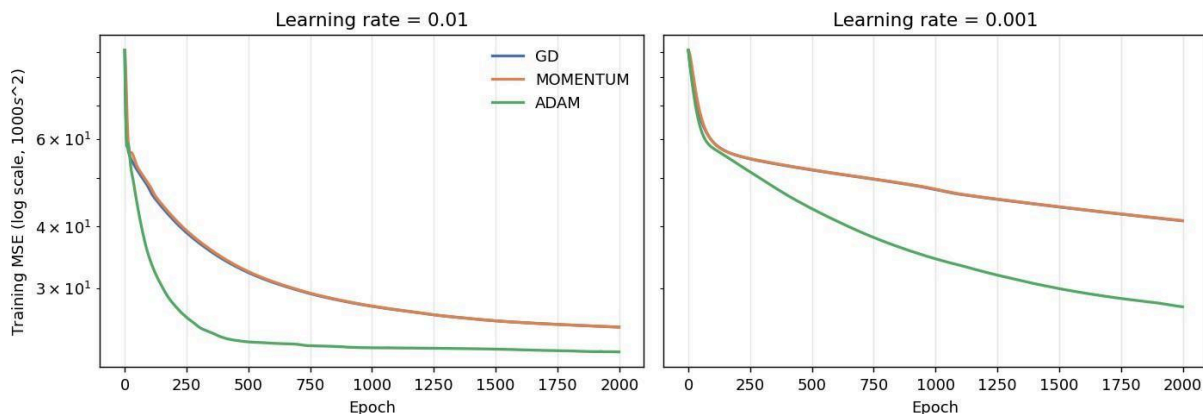


Figure 1. Training MSE vs. epoch (log scale), split by learning rate. At $lr=0.01$ all three optimizers reach a similar loss, with Adam converging fastest; GD and Momentum track almost identically since this small network's loss surface has little curvature for momentum to exploit. At $lr=0.001$, GD and Momentum are still descending slowly after 2000 epochs while Adam is much closer to convergence, since its per-parameter adaptive step size compensates for the small global learning rate.

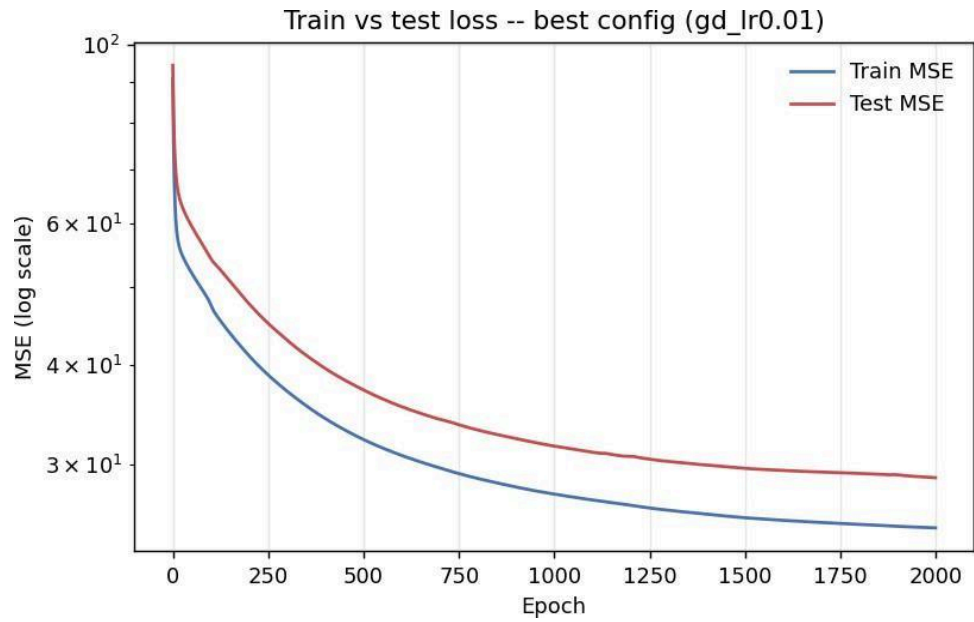


Figure 2. Train vs. test loss for the best configuration (GD, lr=0.01). The train/test gap stays small throughout training, indicating limited overfitting for this small model.

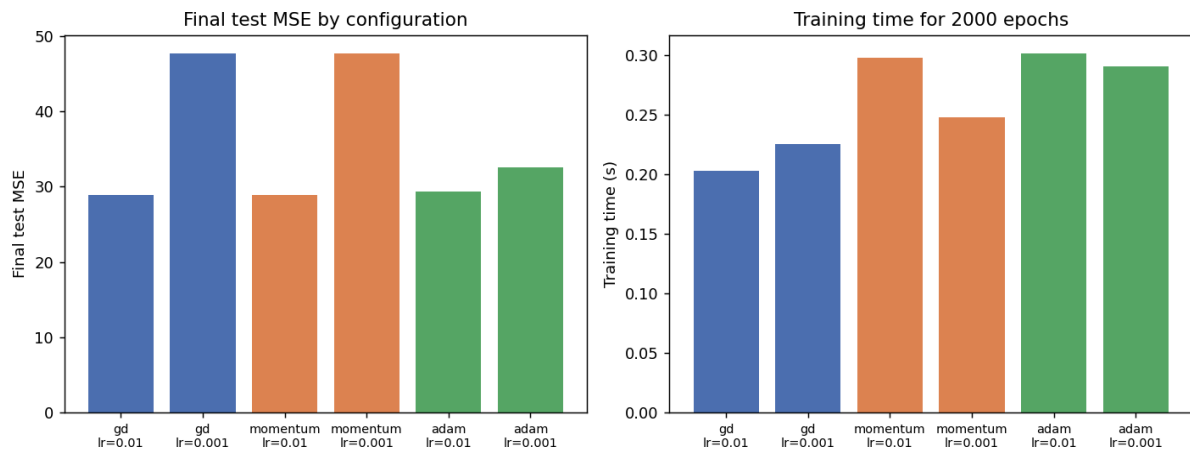


Figure 3. Final test MSE (left) and wall-clock training time for 2000 epochs (right), across all six configurations. Runtime differences are small and mostly reflect Adam and Momentum's extra per-parameter bookkeeping relative to plain GD.

4. Evaluation on the Test Set

The best configuration overall was **GD with learning rate 0.01**, reaching a test MSE of **28.90** (\$1000s)² (RMSE ≈ 5.38 , i.e. typical prediction error of about \$5,376 on home values ranging roughly \$5,000-\$50,000).

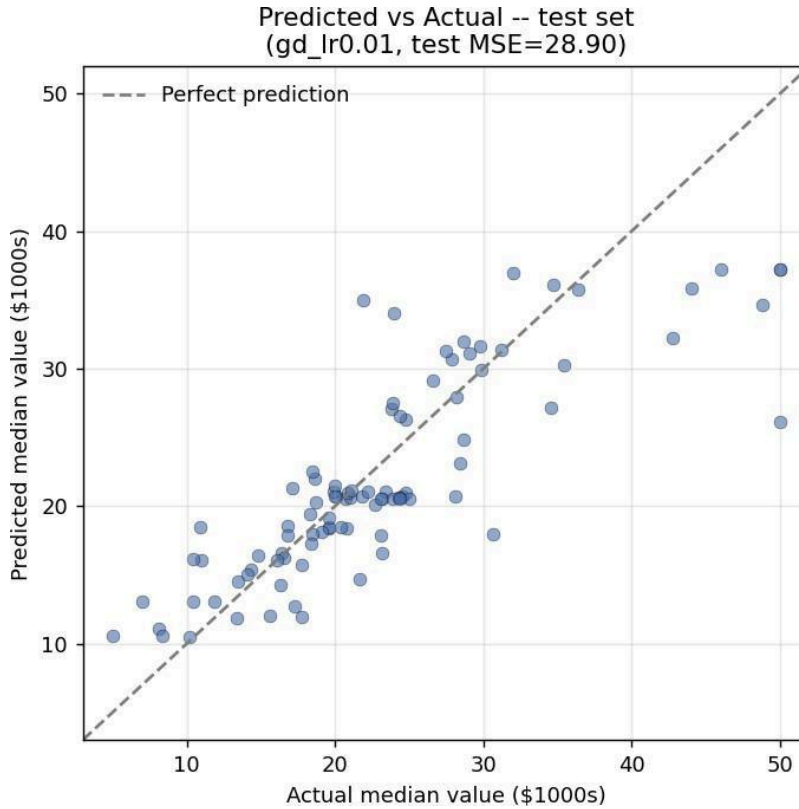


Figure 4. Predicted vs. actual median home value on the test set. Points near the dashed line are accurate predictions. The model captures the overall trend but, using only two features, cannot resolve finer variation -- notably around actual value 50 (\$50,000), where the historical Boston Housing data is known to be top-coded/censored, making that cluster inherently hard to predict.

5. Bonus 1: Additional Hidden Layer

Architecture	Train MSE	Test MSE
2 hidden layers (5-3)	25.03	28.90
3 hidden layers (5-3-2)	25.81	31.01

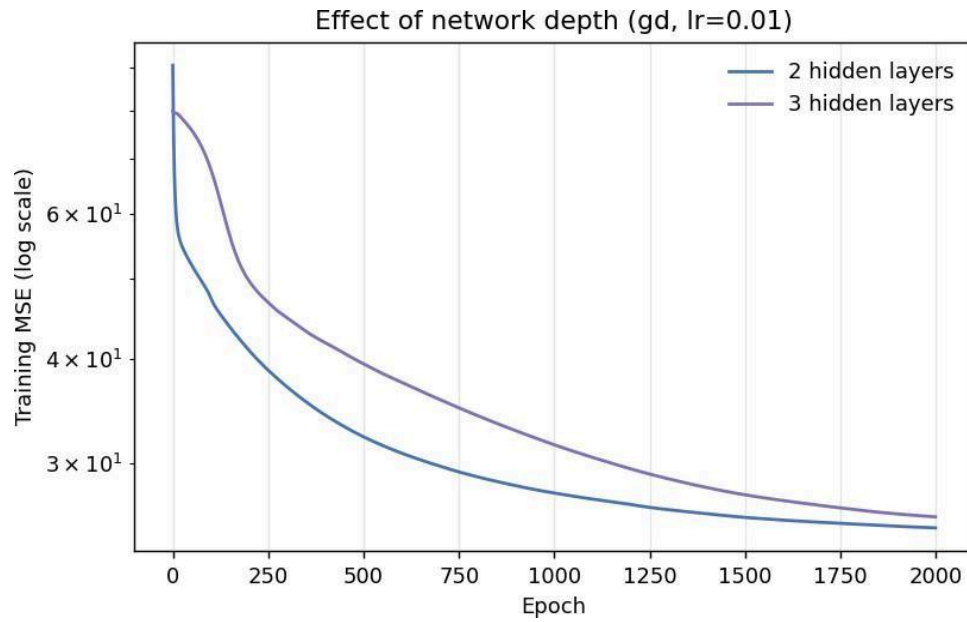


Figure 5. Adding a third hidden layer (2 neurons) does not improve test MSE here; with only two input features the extra layer adds optimization difficulty (more parameters, a longer gradient path) without adding useful representational capacity, and converges slightly slower than the required 2-hidden-layer network.

6. Bonus 2: L2 Regularization

L2 lambda	Train MSE	Test MSE	Test - Train gap
0.0	25.03	28.90	3.87
0.001	25.05	28.93	3.88
0.01	25.30	29.24	3.94
0.1	29.98	31.97	1.98
1.0	79.44	90.64	11.20

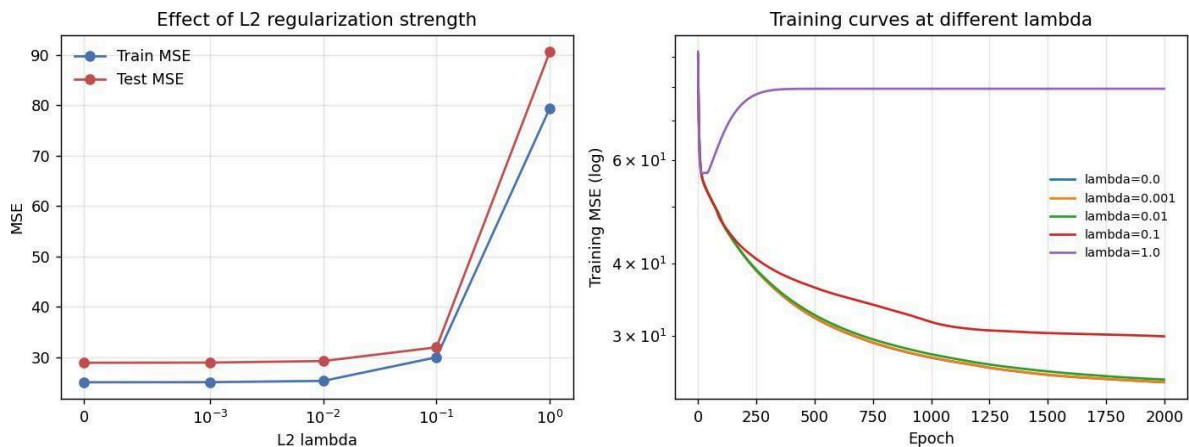


Figure 6. Small L2 penalties ($\lambda \leq 0.01$) barely change train or test MSE, since this small model was not overfitting much to begin with. At $\lambda=1.0$ the penalty dominates the loss, shrinking weights so aggressively that the network can no longer fit even the training data -- both train and test MSE rise sharply, the classic signature of under-fitting from over-regularization. Note the train/test gap actually shrinks at $\lambda=0.1-1.0$, but only because both errors are getting worse together, not because generalization improved.

7. Observations & Discussion

Optimizers & learning rate. At the larger learning rate (0.01), GD, Momentum and Adam converge to similar training loss by epoch 2000; GD and Momentum are nearly indistinguishable because this small network's loss surface is close to convex, leaving little for momentum to exploit. At the smaller learning rate (0.001) the gap is obvious: Adam is much closer to convergence than GD/Momentum, since it adapts an effective per-parameter step size rather than relying on one global rate -- this is exactly the behaviour Adam is designed to produce.

Generalization. The train/test MSE gap for the best configuration is modest (about 4 points of MSE), consistent with a small model (24 learnable weights) applied to a low-dimensional, 2-feature problem -- there is limited capacity for it to overfit.

Depth (bonus). A third hidden layer did not help test MSE; extra depth added optimization difficulty without adding useful capacity, since the input is only 2-dimensional.

L2 regularization (bonus). Regularization strength needs to be tuned carefully: negligible at small λ , clearly harmful once λ is large enough to prevent the network from fitting the training data at all.

Predicted vs. actual. Errors concentrate around the actual-value cluster near \$50,000, a top-coded / censored value in the historical Boston Housing data -- an artifact of the dataset rather than of the model.